

RESOLUCIÓN DEL DISEÑO FUNCIONAL: THE VICTOR CV

- El proyecto **"The Victor CV"** nace de la necesidad de evolucionar el concepto de currículum vitae tradicional hacia una plataforma de marca personal dinámica e interactiva. Utilizando la tecnología **Low-Code de OutSystems**, se ha diseñado una solución que no solo muestra la trayectoria profesional, sino que integra en tiempo real la disciplina personal y los hitos técnicos del autor.
- **La Idea Original**
 - La concepción del proyecto se basa en la transparencia y la bidireccionalidad. La idea es que el reclutador o visitante no sea un espectador pasivo, sino que pueda interactuar con los datos, filtrar proyectos y visualizar el rendimiento real (tanto técnico como deportivo) de Víctor Navarrete.
- **Esquema Conceptual (Brainstorming)**
 - Para la definición de la navegación y la experiencia de usuario (UX), se realizó un diseño preliminar donde se establecieron los niveles de acceso y la distribución de la información en cuatro ejes principales.

login -que se pueda entrar siendo anonimo - que se pueda entrar solo con el nombre -que se pueda entrar iniciando sesion	sobre mi links a las otras paginas, inicio de sesión - para contactacion que se pueda poner comentarios de mejora o para contactar i que llegue el correo al mio -proyectos actuales -como voy -donde trabajo/estudio	proyectos -aquí estaría bien que se pudieran ver los proyectos que tengo en el GitHub con un llamado a API - también las otras cualidades como los idiomas o otras habilidades tipo el stack tecnologico	aquí iran certificados , títulos, otras cosas sobre mi
--	---	---	---

2. BASE DE DATOS INICIAL (BBDD)

- Explicación del modelo de datos base con el que arranca el proyecto (Core Services).
- Diagrama o listado de las primeras entidades creadas (ej. [AccessLog](#), [Candidate](#)).

3. MAPA DE PANTALLAS (UI) (sera manual de usaurio cuando este todo acabado)

Explicación visual de cada pantalla y su propósito. Los detalles técnicos se enlazan a la sección de funcionalidades.

- **3.1 Pantalla 1: Login / Puerta de Acceso**
 - Descripción de la pantalla.
 - *Links a lógica:* [ Ver implementación de Acceso Anónimo e Identificado (Fecha)]
 - **3.2 Pantalla 2: Sobre Mí & Estado Actual**
 - Descripción de la pantalla.
 - *Links a lógica:* [ Ver implementación de Dashboard Salud (Fecha)], [ Ver envío de Email (Fecha)]
 - **3.3 Pantalla 3: Proyectos & Stack Técnico**
 - Descripción de la pantalla.
 - *Links a lógica:* [ Ver integración API GitHub (Fecha)]
 - **3.4 Pantalla 4: Archivo & Certificaciones**
 - Descripción de la pantalla.
 - *Links a lógica:* [ Ver integración descargas Drive (Fecha)]
-

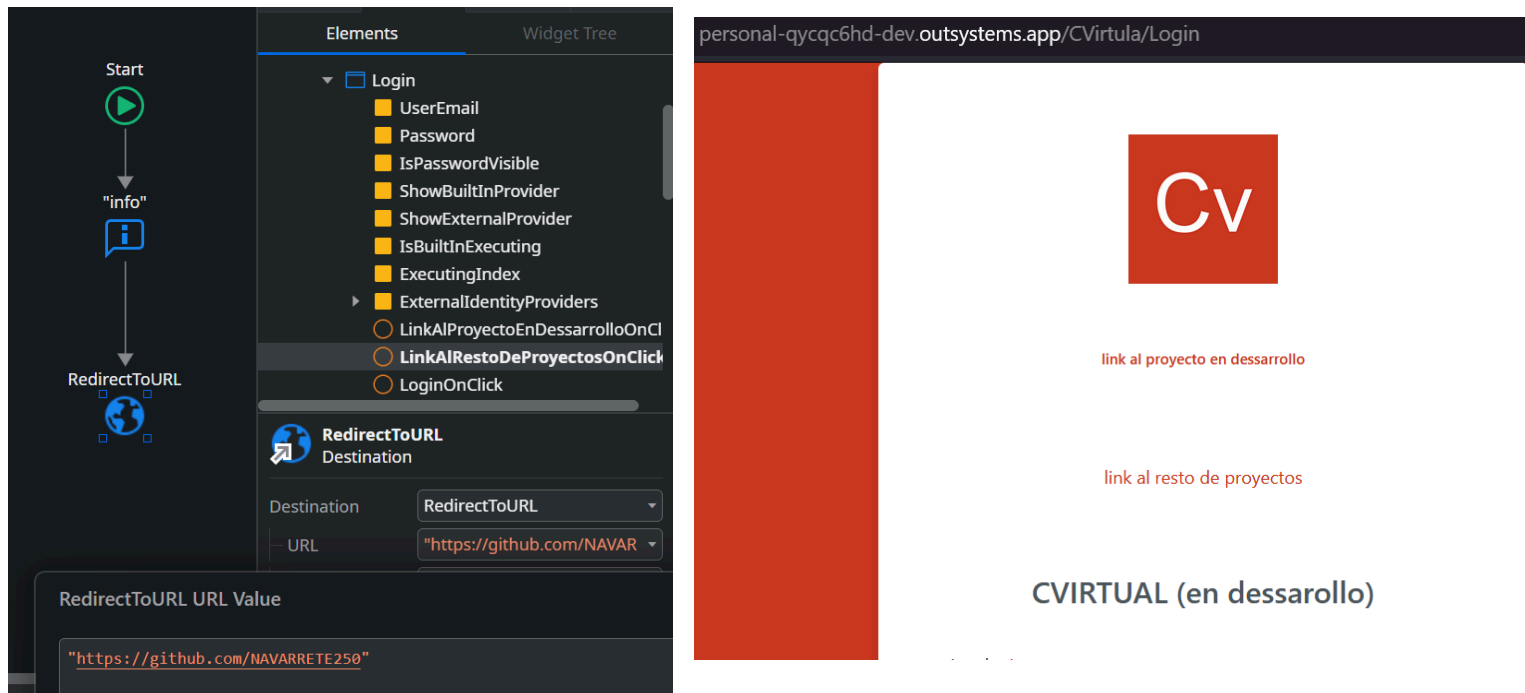
4. HISTORIAL DE FUNCIONALIDADES Y LÓGICA (Por Fecha)

[Fecha: 07/03/2026] - Funcionalidad 1: Redirección Temporal a Repositorios Externos

Debido a que la aplicación se encuentra en fase de desarrollo, se ha implementado una lógica de redirección inmediata para no interrumpir la experiencia del reclutador.

- **Descripción:** Se han añadido dos hipervínculos en la pantalla de Login que redirigen al usuario a los repositorios de GitHub.
- **Lógica Técnica:** Uso de widgets de tipo "Link" con destino a *External URL*. Se ha mantenido la estética del **Stack Tecnológico** definido en el CV.
+1
- **Evidencia Técnica:**
 - *Imagen 1 (Interfaz):* (Aquí pones la captura que me has pasado).

- *Imagen 2 (Lógica):* (Aquí pones la captura de Service Studio mostrando las propiedades del link o el flujo).



[09/03/2026] - Funcionalidad 2: Maquetación de Interfaz y Estructura de Navegación (V2)

En esta fase se ha trabajado en la identidad visual de la aplicación y en la definición del mapa de navegación interno, preparando el terreno para la lógica de datos posterior.

A. Descripción del desarrollo realizado:

1. Rediseño de la UI de Acceso:

- Se ha sustituido la pantalla inicial por un diseño limpio y corporativo con fondo blanco y el logotipo "Cv" en naranja.
- Se han añadido visualmente los campos de Login y Password, además de tres puntos de interacción: un botón principal para "**Crear usuario**", un botón secundario para "**Anonimos**" y el enlace de recuperación de contraseña.

2. Arquitectura de Navegación (Screen Map):

- Se ha creado la estructura de pantallas necesaria para el Hub: **Principal**, **Certificados**, **Proyectos** y **CreacionUsuarios**.
- Se ha implementado la navegación mediante links. Ahora, al pulsar los botones de la portada, la aplicación redirige físicamente a las otras secciones.
- **Organización de Contenidos:** Se han movido los links externos (GitHub) desde la portada hacia la pantalla específica de **Proyectos**, manteniendo así al usuario dentro de la plataforma propia.

3. Preparación de la interfaz Admin:

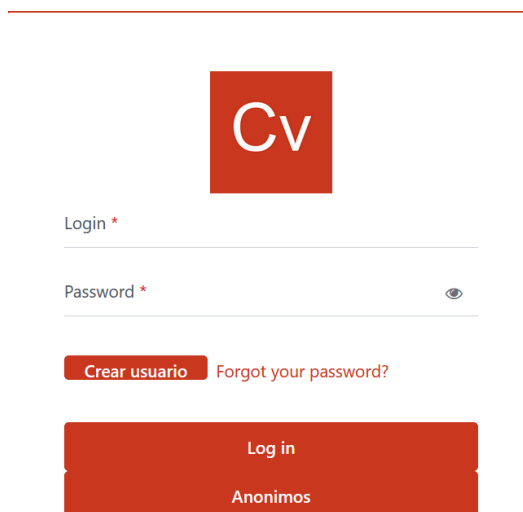
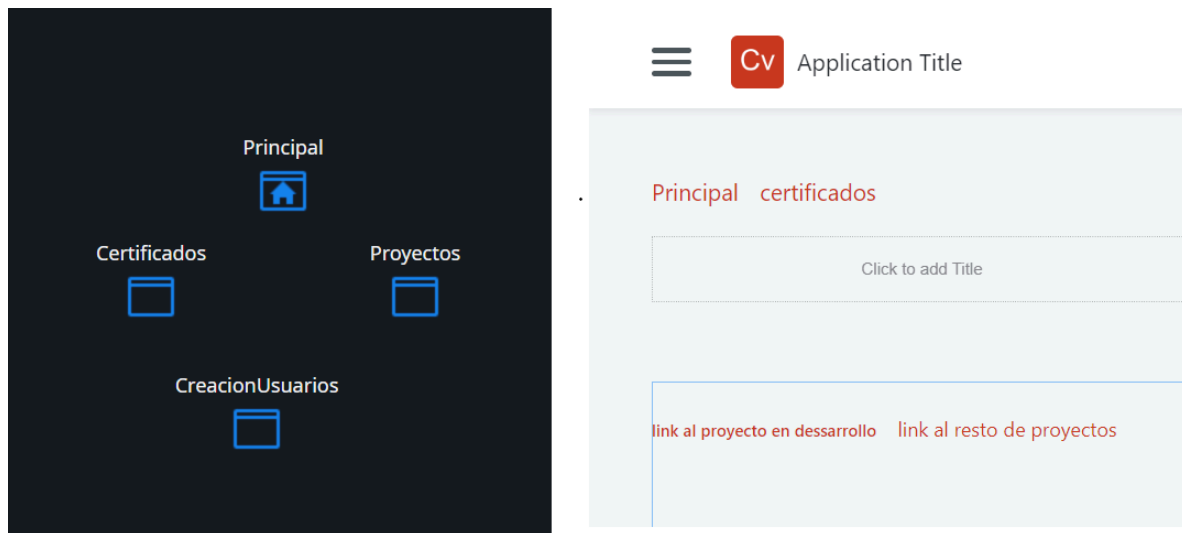
- Se ha creado el acceso visual a la screen de "**Creacion de Usuarios**", quedando esta lista para recibir la lógica de registro en la siguiente fase.

B. Ajustes en la Base de Datos (BBDD):

- No se han realizado cambios en las tablas hoy; el trabajo ha sido 100% de Interfaz (UI) y Flujo de Navegación.

C. Evidencia Técnica (Capturas):

- **Imagen 1 (Nueva Portada):** Captura de la pantalla de inicio con el logo "Cv" y los nuevos botones.
- **Imagen 2 (Mapa de Navegación):** Captura de Service Studio donde se ven las 4 pantallas conectadas y el menú de navegación.

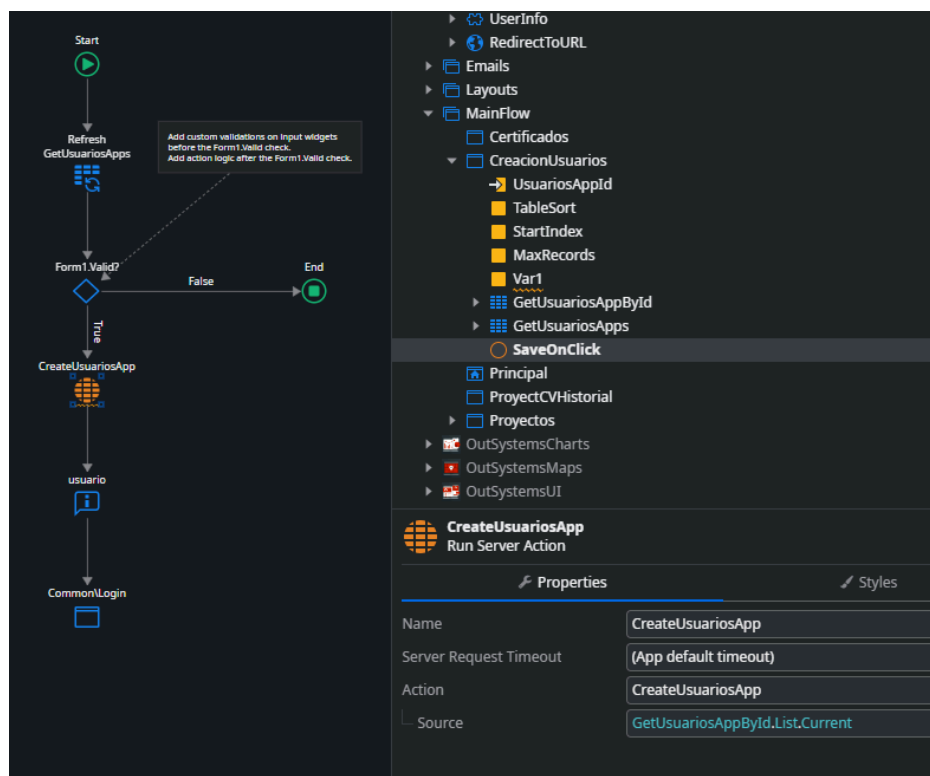
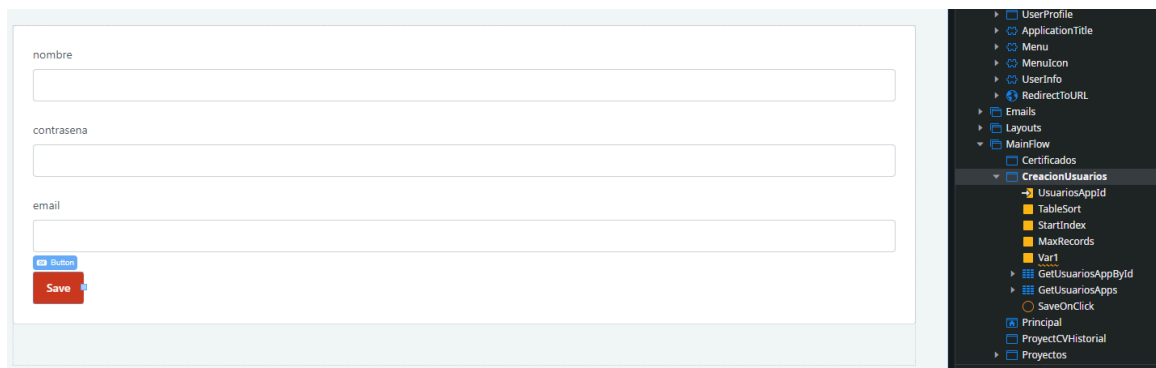


[18/03/2026] - Funcionalidad 3: Sistema de Autenticación Personalizado y Gestión de Usuarios (V3)

En esta fase se ha abandonado el sistema de usuarios nativo de OutSystems para implementar una **lógica de acceso propia**, permitiendo un control total sobre el registro y la validación de credenciales en el Hub.

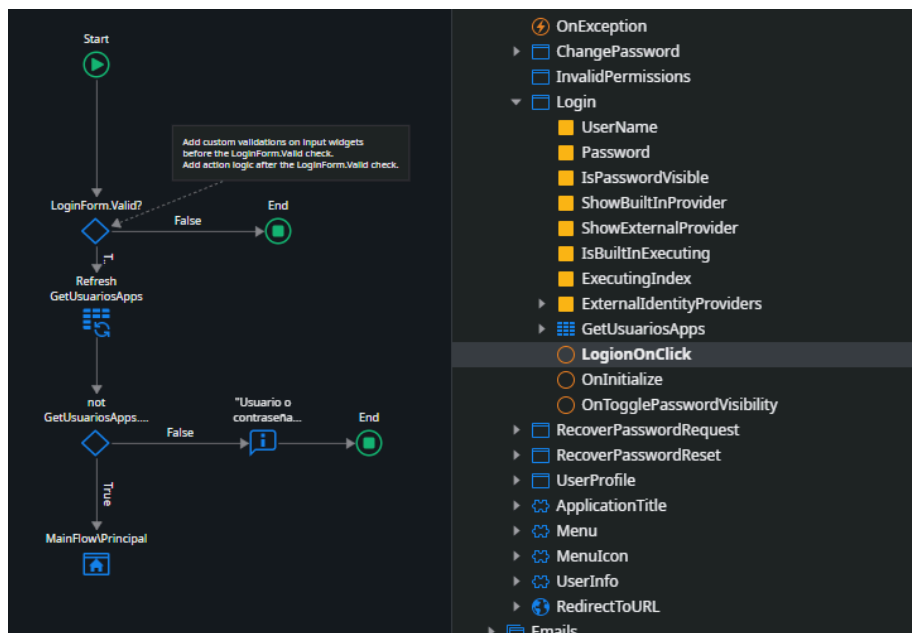
A. Descripción del desarrollo realizado:

- **Creación de Motor de Registro Autogestionado:**
 - Se ha desarrollado la lógica completa en la pantalla **CreacionUsuarios**. Ahora, el botón "Save" valida los datos del formulario y los inserta físicamente en la base de datos mediante la acción **CreateUsuariosApp**.
 - Se ha implementado un flujo de retorno automático que redirige al usuario a la pantalla de Login una vez completado el registro.



- **Lógica de Login Dinámico:**

- Se ha sustituido la validación por defecto por un **Aggregate filtrado (GetUsuariosApps)**. Este buscador compara en tiempo real el **UserName** y la **Password** introducidos con los registros de la tabla personalizada.
- **Control de Acceso:** Se ha configurado el nodo *If* con la condición **not GetUsuariosApps.List.Empty**. Si los datos coinciden, el sistema permite el paso a la pantalla **Principal**; de lo contrario, lanza un mensaje de error visual.



- **Flexibilidad de Acceso (UX):**

- Se ha habilitado un flujo de **Acceso Anónimo** para permitir que visitantes consulten el Hub sin necesidad de cuenta, diferenciando claramente los puntos de entrada en la interfaz de portada.

B. Ajustes en la Base de Datos (BBDD):

- **Entidad UsuariosApp:** Se ha diseñado e implementado una tabla específica con los atributos **Id**, **nombre**, **contrasena** y **email**.
- **Independencia de Datos:** Esta estructura permite que la aplicación gestione sus propios usuarios sin depender de la tabla **User** del sistema, facilitando futuras migraciones o cambios en el perfil del usuario.

UsuariosApp	UsuariosApp	UsuariosApp	UsuariosApp
Id	nombre	contraseña	email
34	jacinto	qwer	
2	juan	1234	
1	manolo	tres	

[18/03/2026] - Funcionalidad 4: Implementación de Bitácora de Proyecto y Trazabilidad (V4)

En esta fase se ha creado una sección dedicada al seguimiento del desarrollo, centralizando toda la documentación técnica, cambios en la lógica y el historial de versiones dentro de la propia aplicación.

A. Descripción del desarrollo realizado:

- **Creación de la Screen ProjectCVHistorial:**
 - Se ha diseñado una nueva interfaz técnica encargada de mostrar el progreso cronológico del Hub.
 - Esta pantalla sirve como **repositorio de conocimiento**, donde se detallan los desafíos técnicos superados (como el sistema de usuarios o los intentos de cifrado).
- **Centralización de Funcionalidades:**
 - Se ha establecido este espacio como el "diario de abordó" del desarrollador, permitiendo que las futuras actualizaciones de la base de datos o de la interfaz queden registradas visualmente para el usuario Admin.
- **Estructura de Navegación Actualizada:**
 - Se ha integrado la nueva pantalla en el mapa global del sitio, asegurando que sea accesible desde el flujo principal de trabajo.

B. Ajustes en la Base de Datos (BBDD):

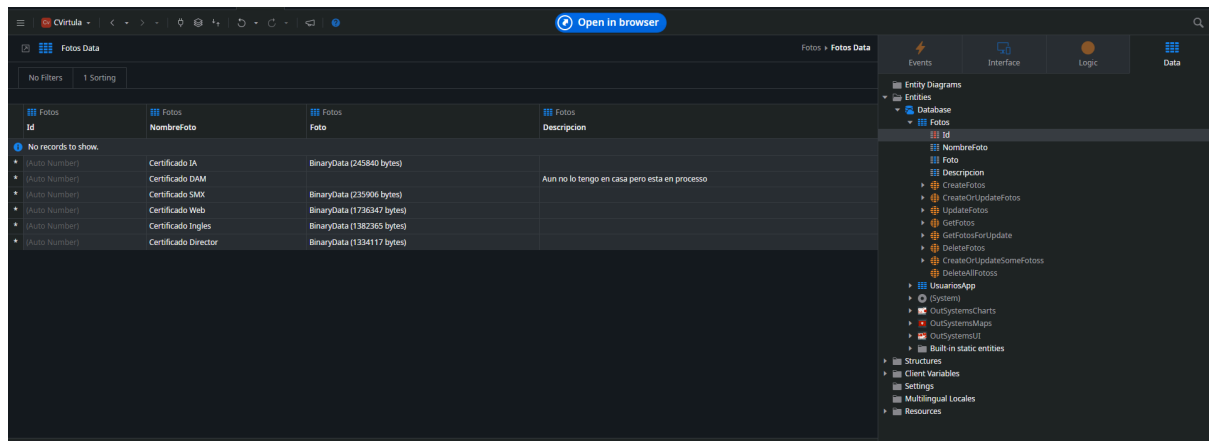
- No se han requerido nuevas entidades para esta pantalla; el contenido se gestiona de forma estática para garantizar una carga rápida y un acceso directo a la información.

[19/03/2026] - Funcionalidad 5: Sistema de Gestión de Títulos y Certificaciones (V5)

En esta fase se ha transformado la sección de "Certificados" de una galería estática a un **módulo dinámico basado en datos**, permitiendo una presentación profesional, escalable y ordenada de la trayectoria formativa.

A. Descripción del desarrollo realizado:

- **Arquitectura de Datos (Backend):**
 - Creación de la entidad **Fotos** en la base de datos, con atributos para **Id** (Autoincremental), **NombreFoto**, **Foto** (Binary Data) y **Descripcion**.
 - Carga inicial de 6 registros reales, incluyendo certificaciones de IA, Desarrollo Web, Inglés y formación en tiempo libre.



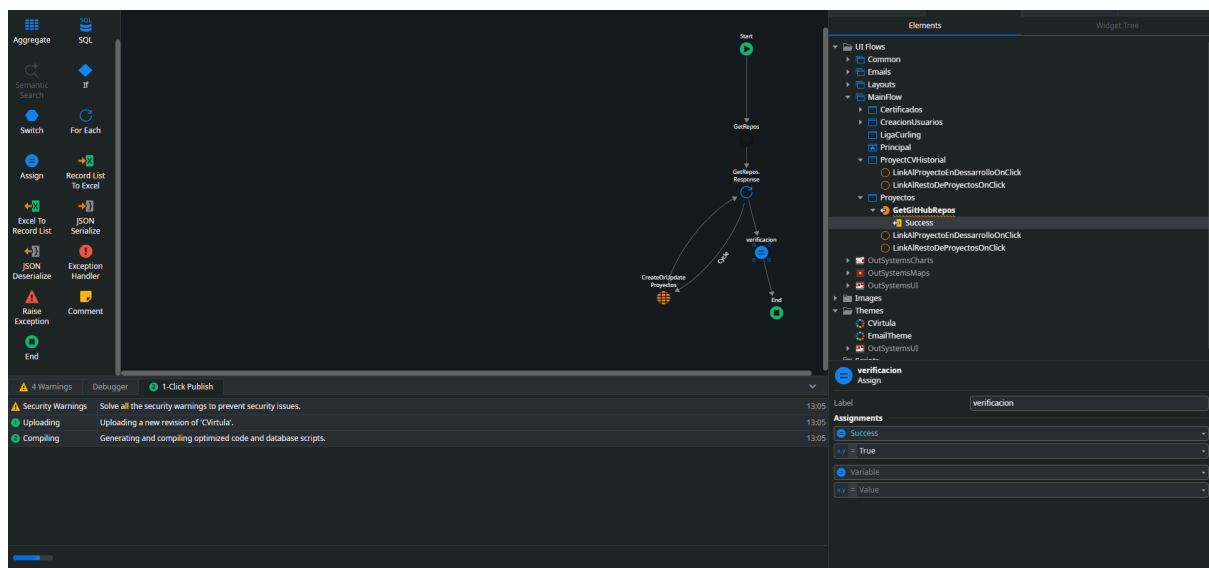
- **Optimización de la Interfaz (UI/UX):**
 - **Migración a Listado Vertical:** Se ha sustituido el widget *Gallery* por un widget *List* para mejorar la legibilidad y permitir descripciones largas al lado de cada imagen.
 - **Diseño de Cards Profesionales:** Implementación de tarjetas contenedoras con sistema de columnas para separar el contenido visual (título/diploma) del contenido textual (explicación técnica).
- **Lógica de Ordenación:**
 - Configuración de un **Aggregate (GetFotos)** con ordenación ascendente por **Id**, garantizando que el historial siga una secuencia lógica y controlada por el desarrollador.

B. Ajustes en la Base de Datos (BBDD):

- **Entidad **Fotos**:** Configurada con **Is Autonumber = Yes** para automatizar el índice de certificaciones.

- **Gestión de Nulos:** Preparación del registro para el título de **DAM** (Desarrollo de Aplicaciones Multiplataforma), configurado actualmente como "En proceso" para mantener la trazabilidad del CV mientras se obtiene el título oficial.

CV CVirtula	Login
Principal Proyectos CVirtual	
Certificado IA 	Formación especializada en "Desarrollo con IA: de 0 a Producción" (BIG School), cubriendo flujos de trabajo con LLMs y herramientas de IA generativa.
Certificado DAM	(Aun no lo tengo en casa pero esta en proceso) Grado Superior en Desarrollo de Aplicaciones Multiplataforma. Enfoque en desarrollo de software, bases de datos y sistemas ERP.
Certificado SMX 	Título de Técnico en Sistemas Microinformáticos y Redes. Especialización en montaje, mantenimiento de equipos, redes locales y servicios en red.
Certificado Web 	Certificado de profesionalidad en Confección y Publicación de Páginas Web. Enfoque en maquetación HTML/CSS, diseño responsive y publicación de sitios.
Certificado Ingles 	Acreditación de nivel Pre-Intermediate A2.2 por International House Dublin, centrado en competencia comunicativa y gramática aplicada



Resumen del Problema y Resolución

Problema inicial

Al intentar asignar la fuente de datos (*Source*) a un widget **List** en la pantalla *Proyectos* de OutSystems, no aparecía directamente el método REST **GetRepos** ni la API consumida de GitHub.

Causa

En OutSystems, los métodos REST de backend o Server Actions no se pueden conectar directamente a los widgets de la interfaz gráfica. Necesitan ser consumidos a través de un **Data Action** en el ámbito de la propia pantalla para procesar y exponer los datos hacia la vista.

Pasos ejecutados para resolverlo:

1. Configuración del Data Action en la pantalla:

- Se creó un **Data Action** (*Fetch Data from Other Sources*) dentro de la pantalla **Proyectos**.
 - Se arrastró la acción de integración REST **GetRepos** de GitHub al flujo lógico (entre el nodo **Start** y **End**).
2. **Creación del parámetro de salida (Output Parameter):**
- Se añadió un **Output Parameter** al Data Action (ej. **RepoList**).
 - Se le asignó el tipo de datos (*Data Type*) de lista correspondiente a la respuesta de la API: **GetReposResponse List**.
3. **Mapeo de datos en el flujo:**
- Se añadió un nodo **Assign** entre la llamada REST y el **End**.
 - Se asignó el valor retornado por la integración al Output Parameter:
RepoList = GetRepos.Response.
4. **Vinculación con la interfaz visual (UI):**
- En las propiedades del widget **List** de la pantalla *Proyectos*, se asignó en la propiedad **Source** la variable de salida expuesta por el Data Action (**DataAction.RepoList**).
 - Se añadieron los campos deseados dentro del ítem de la lista (como **Name**, **Language**, **Html_url**, etc.) usando expresiones vinculadas a **RepoList.Current.<Atributo>**.

Cv

CVirtula

Logir

Auditor https://github.com/NAVARRETE250/Auditor	Kotlin
camaronX https://github.com/NAVARRETE250/camaronX	Kotlin
CobolPracticas https://github.com/NAVARRETE250/CobolPracticas	Un poco de Cobol para practicar COBOL
CVirtual https://github.com/NAVARRETE250/CVirtual	Este repositorio sera actualizaciones de mi CVirtual en Outsystems
DAM-Programacio-main https://github.com/NAVARRETE250/DAM-Programacio-main	Java
ERP-Python	